

LLMs para Devs

Guía práctica para integrar
IA en el ciclo de desarrollo





Introducción

Esta guía explora cómo *LLMs* transforman el flujo de trabajo del desarrollador moderno, desde generación de código hasta agentes autónomos.

Presenta modelos clave, patrones de consumo, flujos prácticos y ejemplos de prompts aplicables en entornos productivos.

01

Panorama de Modelos



GPT-4 (casos y uso)

GPT-4 destaca en *generación creativa* y asistentes conversacionales, útil para diseño de arquitecturas, prototipos y revisión de código.

Se emplea tanto en *borradores* y brainstorming como en integración via API para tareas de completado y documentación automática.

Claude 3 (ventana contexto)

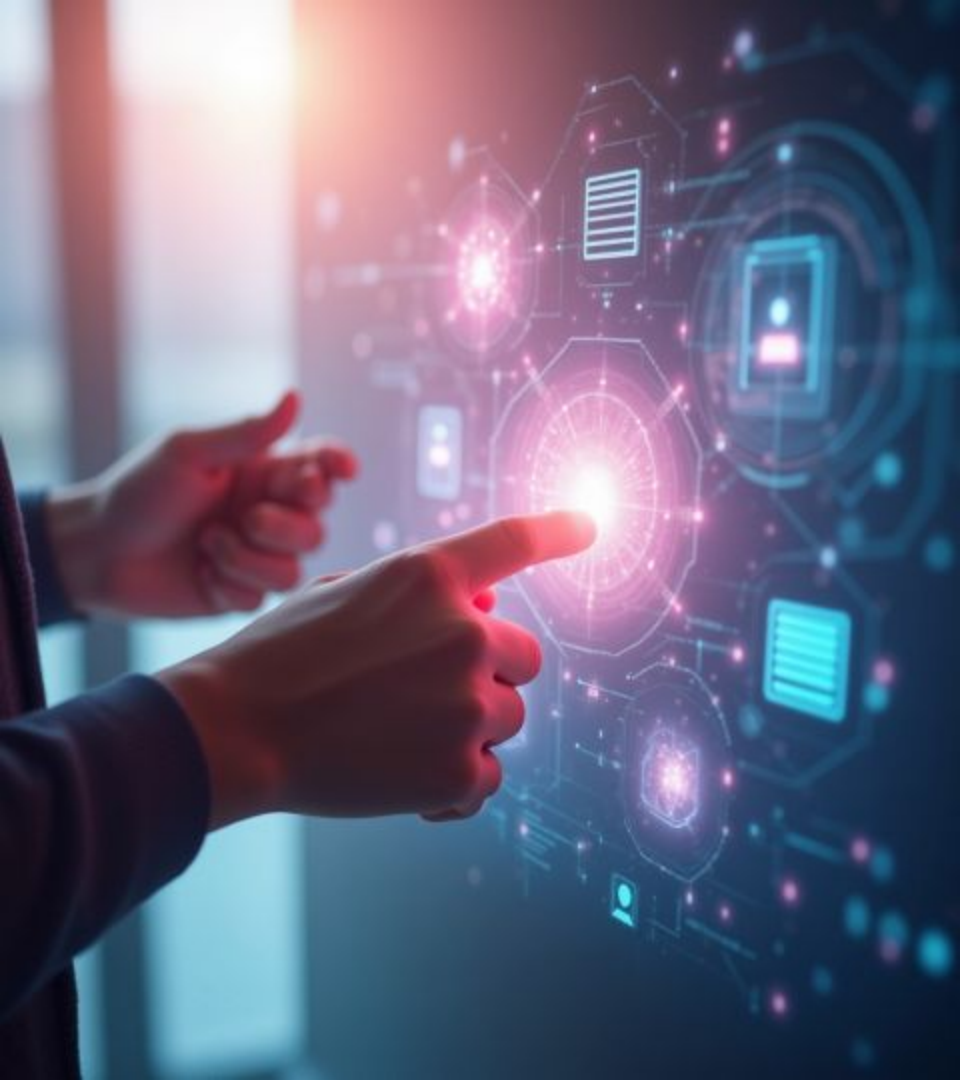
Claude 3 Opus/Sonnet sobresale por su amplia *ventana de contexto*, ideal para análisis de grandes bases de código y prompts extensos.

Útil en revisiones históricas de repositorios, extracción de requisitos y tareas que requieren mantener contexto extenso.



Gemini 1.5

Gemini 1.5 Pro ofrece capacidades *multimodales* y alta integración con el ecosistema Google, ideal para soluciones en la nube que requieren procesamiento de texto, imagen y señales adicionales.



02

Formas de Consumo

Web vs integrado

Las interfaces web (ChatGPT, Claude, Gemini) son excelentes para *brainstorming* y exploración rápida.

Las integraciones embebidas ofrecen productividad continua; elegir según seguridad, latencia y control de datos.



IDEs y Copilot

Integrar LLMs en IDEs (VS Code + GitHub Copilot, Cursor) mejora la productividad con **autocompletado** y sugerencias contextuales.

Mantener revisión humana y configurar límites de confianza para evitar *alucinaciones*.

CLI y agentes

Las herramientas CLI y agentes permiten ejecutar tareas rápidas y automatizadas en el flujo local, ideal para scripts y pipelines.

Los agentes (p. ej. Aider, Cursor agent mode) pueden modificar archivos, ejecutar pruebas y orquestar pasos con **supervisión humana** obligatoria.

02

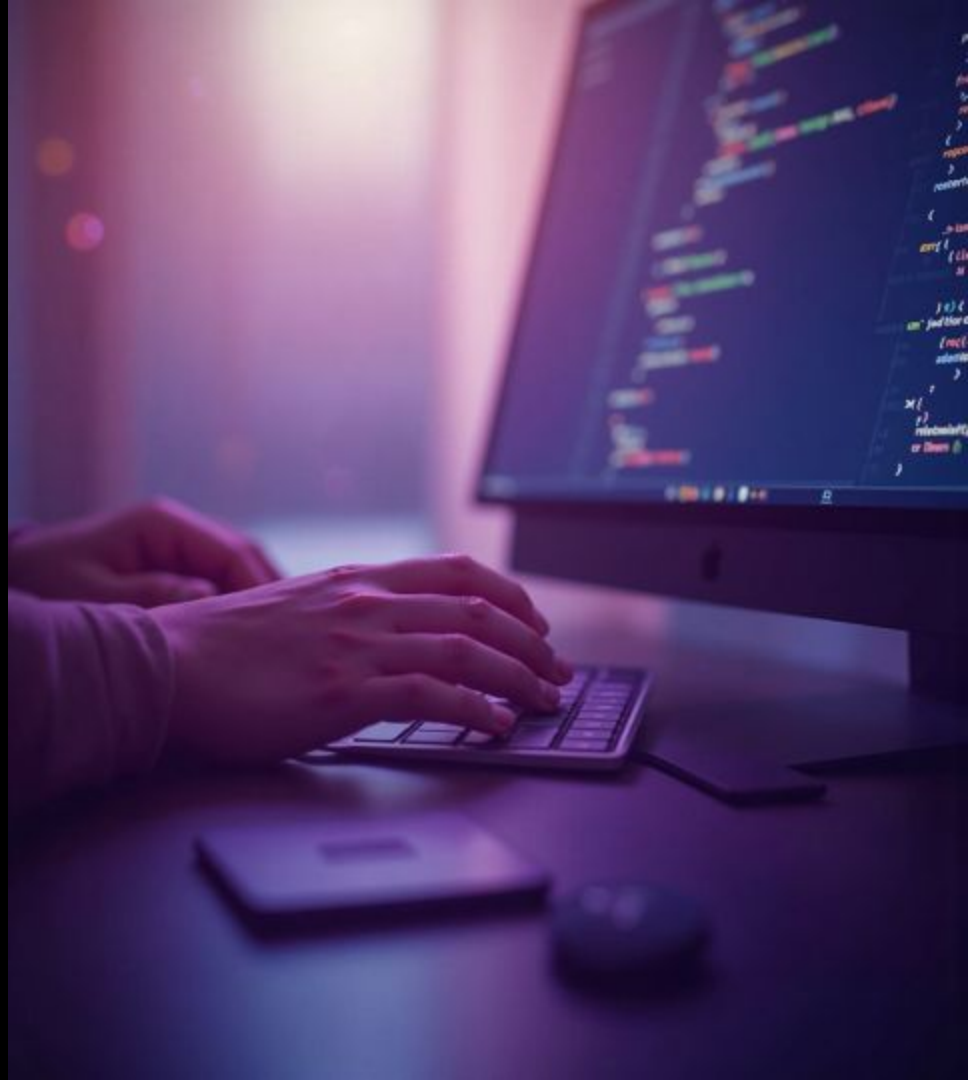
Flujos de Trabajo



Generación y autocompletado

Los LLMs aceleran la creación de *boilerplate* y funciones completas, reduciendo tiempo en tareas repetitivas.

Integrar completado contextual en el IDE mejora la productividad, siempre con revisión de calidad y seguridad.



Refactorización y debugging

La IA identifica patrones, reduce complejidad y propone correcciones para errores lógicos complejos.

Usar prompts dirigidos para explicar razones del cambio y validar rendimiento antes de aplicar refactors.

Documentación y tests

Generación automática de *docstrings*, README y suites de tests (unitarios/integración) ahorra tiempo y mejora cobertura.

Asegurar que tests generados cubran edge cases y sean revisados por el equipo.

03

Práctica y Riesgos





Prompts ejemplares (5)

Ejemplos prácticos listos para usar: refactorización Python, debugging Java, tests Jest, migración React→TS y consulta SQL PostgreSQL.

Cada prompt debe incluir contexto, fragmento de código y criterios de éxito para resultados útiles.

Agentes autónomos

Los agentes pueden ejecutar flujos completos (modificar, probar, desplegar) pero requieren límites claros y logs auditables.

Preferir modos *sandbox* y permisos mínimos al integrar agentes en entornos productivos.

Riesgos y buenas prácticas

Proteger datos sensibles: nunca subir llaves ni código propietario sin encriptar. Verificar outputs por alucinaciones.

Establecer revisiones humanas, validaciones automáticas y políticas de seguridad para despliegues con LLMs.



Conclusiones

Los LLMs transforman el ciclo de desarrollo habilitando productividad y calidad, desde generación hasta agentes autónomos.

Adoptar modelos según privacidad y coste, usar *prompts* precisos y aplicar controles de seguridad y revisión humana.

Gracias!

Preguntas?

CREDITS: This presentation template was created by [Slidesgo](#), and includes icons, infographics & images by [Freepik](#)

Please keep this slide for attribution